პროფესიული საგანმანათლებლო პროგრამა: Front-end პროგრამირება

მოდული: პრაქტიკული პროექტი (რეგისტრაციის № 06.1.1.9)

საბოლოო ტექნიკური ანგარიში

- **სტუდენტი:** ნათია გოროზია
- **შესრულების ფორმატი:** ინდივიდუალური
- **პროექტის დასახელება და სფერო:** B2B Logistics Dashboard (ოპერაციული ლოჯისტიკისა და საწყობების მართვის სისტემა)
- **GitHub რეპოზიტორიის ბმული:** https://github.com/natatogorozia-pixel/dashboard-app

1. სამუშაოს დაგეგმვა და ორგანიზება

1.1. სამუშაო გეგმა, ეტაპები (Milestones) და რისკების განსაზღვრა

- **პროექტის მიზანი და ბიზნეს-ამოცანა:**

  პროექტი წარმოადგენს თანამედროვე B2B ლოჯისტიკურ პლატფორმას, რომელიც უზრუნველყოფს საწყობების, სატრანსპორტო ნაკადებისა და ფინანსური ინდიკატორების ცენტრალიზებულ და ინტერაქციულ მართვას (CRUD). აპლიკაცია წყვეტს მონაცემთა გაფანტულობისა და გვერდის განახლებისას (Refresh) ინფორმაციის დაკარგვის პრობლემას, რის გამოც ის შეუცვლელია ლოჯისტიკური მენეჯერებისა და ადმინისტრატორებისთვის.

- **განხორციელების ეტაპები და ვადები (Milestones):**
  - **ეტაპი I (კვირა 1):** მოთხოვნების ანალიზი, ბიზნეს-კონცეფციის შემუშავება, ტექნოლოგიური სტეკისა და არქიტექტურის დაგეგმვა.
  - **ეტაპი II (კვირა 2–3):** სემანტიკური მარკირება, დიზაინ-სისტემის აწყობა გლობალური CSS ცვლადებით, ძირითადი UI კომპონენტების (`StatCard`, `Modal`) შექმნა.
  - **ეტაპი III (კვირა 4–5):** ინტერაქტიული ფუნქციონალისა და CRUD ოპერაციების ინტეგრაცია, Zustand გლობალური სთეიტის მიბმა `localStorage`-თან, Zod და React Hook Form ვალიდაცია.
  - **ეტაპი IV (კვირა 6):** წარმადობის ოპტიმიზაცია (`useDebounce`, `useMemo`), დაცვითი მექანიზმების (`ErrorBoundary`) დანერგილირება, Cross-Browser ტესტირება და README დოკუმენტაციის მომზადება.
- **იდენტიფიცირებული რისკები და მათი პრევენცია:**

  1. **რისკი 1 (მონაცემთა დაკარგვა Refresh-ისას):** ბრაუზერის გადატვირთვისას ადგილობრივ `useState`-ში შენახული მონაცემების წაშლა ➔ *პრევენცია:* ინტეგრირებული იქნა Zustand-ის `persist` მიდლვეარი, რომელიც ავტომატურად ასინქრონიზებს სთეითს `localStorage`-თან.

2. **რისკი 2 (ფორმის ორმაგი გაგზავნა / Double-Click):** მომხმარებლის მიერ შენახვის ღილაკზე სწრაფი ორმაგი დაჭერით გამოწვეული დუბლირებული მონაცემების ჩაწერა ➔ *პრევენცია:* დაინერგა `isSubmitting` მდგომარეობა, რომელიც აბლოკირებს ღილაკს მოთხოვნის მიმდინარეობისას.

1.2. რესურსებისა და მეთოდების შერჩევა

- **გამოყენებული ტექნოლოგიური სტეკი და არჩევის დასაბუთება:**
  - **Core & Build:** React & Vite — უზრუნველყოფს სწრაფ ასაწყობ გარემოს, მოდულურ არქიტექტურას და მაღალ წარმადობას.
  - **Styling & UI:** Custom CSS (Variables, Grid, Flexbox) — მოქნილი და მსუბუქი სტილიზაცია მკაფიო დიზაინ-ტოკენებისთვის.
  - **State & Persistence:** Zustand (`persist` middleware) + LocalStorage — მარტივი და მსუბუქი გლობალური სთეით-მენეჯმენტი Prop Drilling-ის გარეშე.
  - **Validation & Forms:** React Hook Form + Zod — უზრუნველყოფს ფორმების მკაცრ ტიპიზაციასა და Fail-fast ვალიდაციას.
- **სამუშაო ინსტრუმენტები:** VS Code, Git & GitHub, Chrome DevTools, ESLint, Figma.

1.3. პროფესიული პასუხისმგებლობები და კომუნიკაცია

- **როლებისა და პასუხისმგებლობების განაწილება:**

  ინდივიდუალური პროექტის ფარგლებში შევთავსე Full-cycle დეველოპერის ყველა როლი: UI/UX დიზაინის ადაპტაცია, Front-end არქიტექტურის დაპროექტება, Zustand სთეიტის მენეჯმენტი, QA ტესტირება და პროფესიული დოკუმენტირება.

- **კომუნიკაციისა და სამუშაოს კოორდინაციის მეთოდი:**

  სამუშაო პროცესი იგეგმებოდა ეტაპობრივად, ხოლო კოდის ვერსიონირება და ცვლილებების მართვა ხორციელდებოდა Git-ისა და GitHub რეპოზიტორიის მეშვეობით სემანტიკური კომიტების გამოყენებით.

2. პროფესიული ამოცანის დამოუკიდებლად შესრულება

2.1. ვებგვერდის არქიტექტურა და მარკირება (HTML5)

- **საქაღალდეების მოდულური სტრუქტურა:**

  src/

  ├── assets/ # გრაფიკული რესურსები და ოპტიმიზებული სურათები

  ├── components/ # ხელახლა გამოყენებადი UI კომპონენტები (StatCard, ItemForm, Modal, ErrorBoundary)

├── hooks/ # Custom ჰუკები (useModal.js, useDebounce.js)

├── store/ # გლობალური სთეით-მენეჯმენტი (useLogisticsStore.js)

├── App.jsx # მთავარი აპლიკაციის კომპონენტი

├── App.css # გლობალური სტილები და ბადეები

└── main.jsx # შესასვლელი წერტილი

- **სემანტიკური HTML5 და Accessibility (a11y):**

  გამოყენებულია სემანტიკური თეგები (`<header>`, `<main>`, `<section>`, `<nav>`, `<footer>`), რათა უზრუნველყოფილი იყოს სწორი სტრუქტურა ეკრანის მკითხველებისთვის (Screen Readers). დამატებულია `aria-label` ატრიბუტები და კლავიატურით ნავიგაციის მხარდაჭერა Accessibility სტანდარტების დასაცავად.

2.2. ვიზუალური სტილი და რესპონსივობა (CSS3 / Flexbox / Grid)

- **გლობალური სტილების სისტემა:**

  გამოყენებულია `:root` CSS ცვლადები ფერების, შრიფტებისა და დაშორებების (Spacing) ცენტრალიზებული მართვისთვის, რაც უზრუნველყოფს დიზაინის ერთიანობასა და კოდის სისუფთავეს.

- **Flexbox & Grid გამოყენება:**

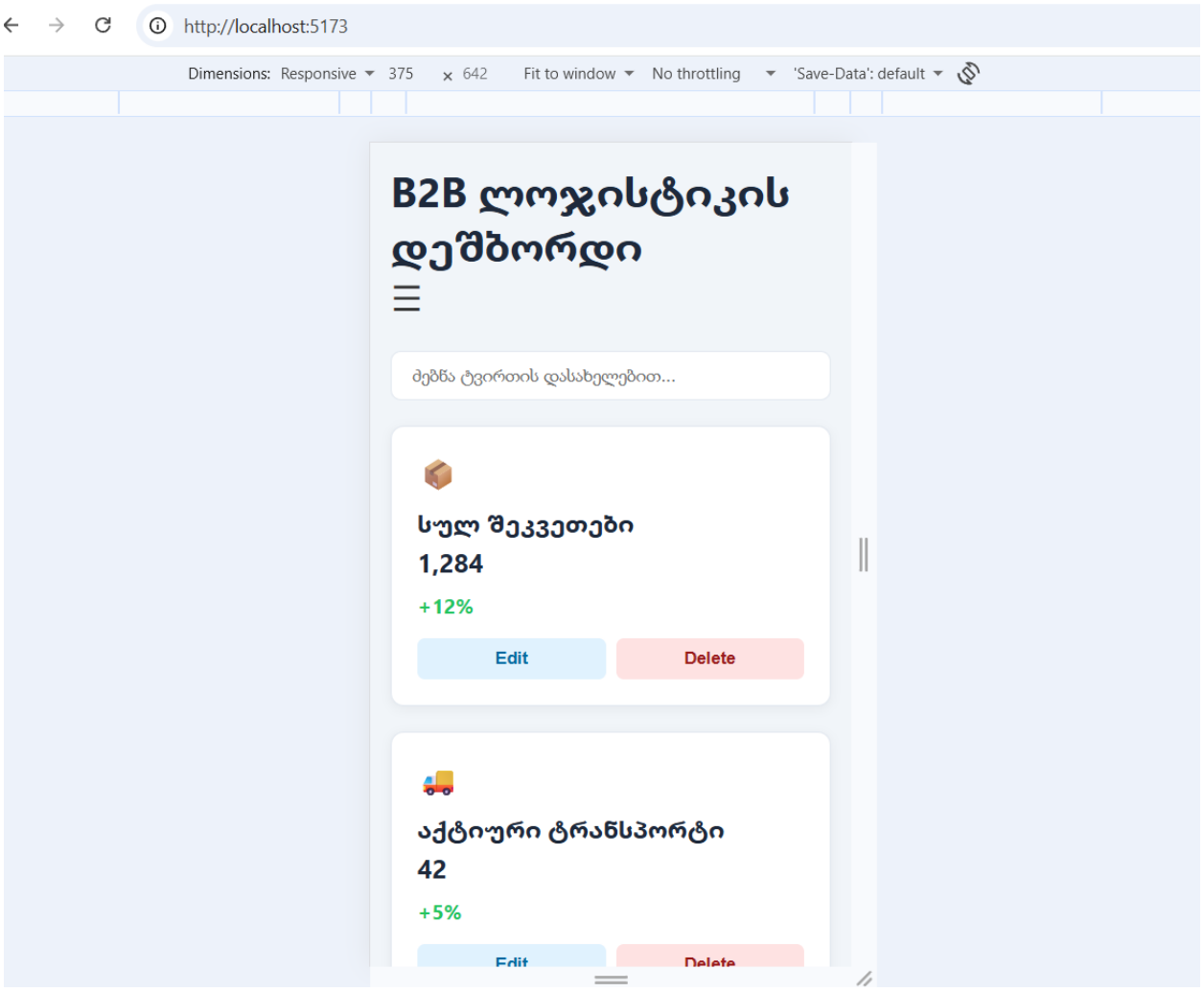
  CSS Grid გამოიყენება დეშბორდის სტატისტიკური ბარათების ზოგადი ბადისთვის (`stats-grid`), ხოლო Flexbox გამოიყენება ჰედერის ელემენტების, მოდალური ფანჯრის სათაურებისა და ღილაკების განსალაგებლად.

- **რესპონსივობა და მობილური მენიუ:**

  ინტერფეისი სრულად ადაპტირებულია დესკტოპ (1440px), პლანშეტ (768px) და მობილურ (375px) ეკრანებზე მედია-ქვერებისა და ჰამბურგერ-მენიუს მეშვეობით.

- ***დესკტოპ და მობილური (375px) ხედის სქრინშოტები***





2.3. ინტერაქტიული ელემენტები და ფუნქციონალი (JavaScript / React)

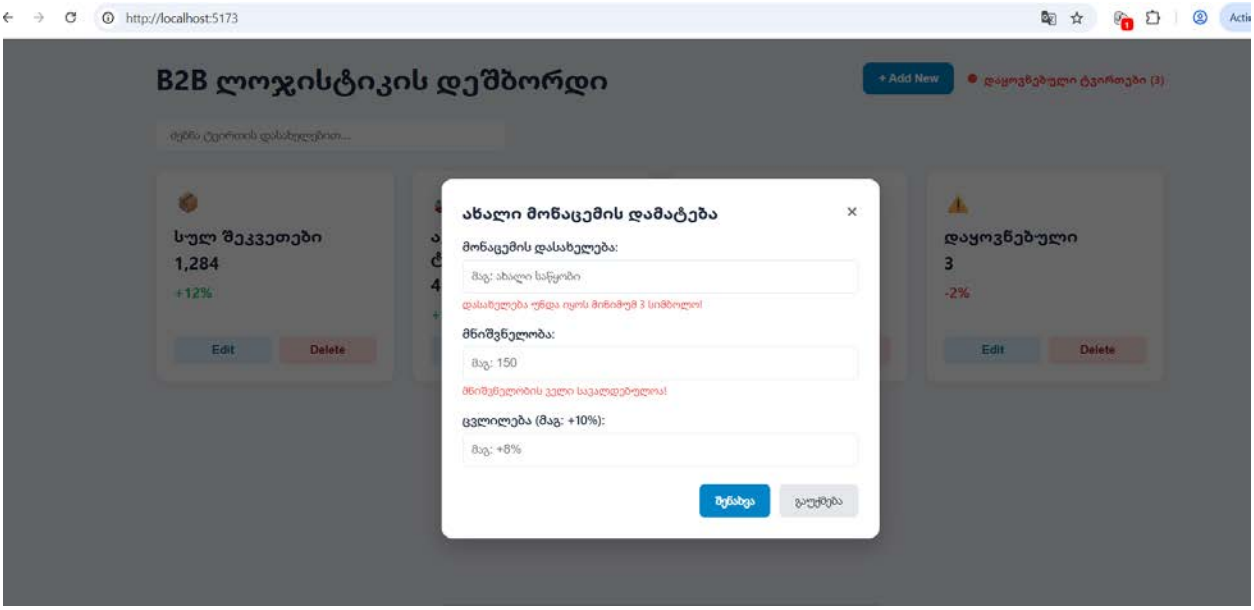
- **CRUD ოპერაციები და სთეითი:**

  სისტემა უზრუნველყოფს მონაცემების სრულ CRUD ციკლს (ახალი ტვირთის/სტატისტიკის დამატება, არსებულის რედაქტირება, წაშლა და დინამიური

წაკითხვა). Zustand Store ცენტრალიზებულად მართავს სთეითს და `persist` მექანიზმით ინახავს `localStorage`-ში.

- **დინამიური ძიება და მოდალები:**

  განხორციელებულია რეალურ დროში ძიება `useDebounce`-ით. მოდალური ფანჯრები რენდერდება `ReactDOM.createPortal`-ის გამოყენებით პირდაპირ `document.body`-ში, რაც ხსნის მშობელი კონტეინერების CSS შეზღუდვებს (`overflow: hidden`). ფორმების ვალიდაცია ხორციელდება Zod სქემითა და React Hook Form-ით.



2.4. ვერსიების კონტროლი (Git / GitHub)

- **Git კონფიგურაცია და კომიტები:**

  შემუშავებულია სწორი `.gitignore` ფაილი (რომელიც ზღუდავს `node_modules/` და აგებული ფაილების ატვირთვას). პროექტი იმართება ბრენჩების სტრუქტურითა და Conventional Commits სტანდარტით (`feat:`, `fix:`, `refactor:`, `docs:`).

2.5. ოპტიმიზაცია და პრობლემების გადაჭრა (Optimization & Troubleshooting)

- **წარმადობა და დაცვითი მექანიზმები:**
  - გამოყენებულია `useMemo`, `useCallback`, `React.memo` და `useDebounce` ზედმეტი რენდერების ასაცილებლად.
  - დაინერგა `ErrorBoundary` თეთრი ეკრანის თავიდან ასაცილებლად და ფორმის გაგზავნისას ღილაკის ბლოკირება (`isSubmitting`).

- **გადაჭრილი ტექნიკური გამოწვევები (Bug / Challenge):**

  გამოწვევა: ადგილობრივი სთეითიდან Zustand გლობალურ სთეითზე გადასვლისას, გვერდის განახლებისას ზოგჯერ ხდებოდა მონაცემთა აცდენა ქეშსა და მიმდინარე ობიექტებს შორის.

  გადაწყვეტა: დაზუსტდა Zustand-ის `persist` მიდლვეარის კონფიგურაცია და დაწესდა მკაცრი ტიპიზაცია Zod სქემების მეშვეობით, რამაც უზრუნველყო მონაცემთა სრული სინქრონიზაცია.

3. შესრულებული ამოცანების ანგარიშგება

3.1. პროექტის ანგარიში (Documentation / README.md)

- **დადასტურება:**

- GitHub რეპოზიტორიაში: https://github.com/natatogorozia-pixel/dashboard-app

  განთავსებულია პროფესიონალური, სრულყოფილი `README.md` ფაილი, რომელიც მოიცავს პროექტის აღწერას, ძირითად ფუნქციონალს, ტექნოლოგიური სტეკის ცხრილს, საქაღალდეების სტრუქტურას, ლოკალურად გაშვების ინსტრუქციას (`git clone ➔ npm install ➔ npm run dev`) და სატესტო ანგარიშებს.

3.2. პრეზენტაცია და პროდუქტის წარდგენა (Demo Scenario & Roadmap)

- **სადემონსტრაციო სცენარი (Happy Path — 5 წუთიანი Live Demo):**
  1. **შესავალი (1 წთ):** პროექტის მიზნისა და ბიზნეს-კონცეფციის მოკლე მიმოხილვა.
  2. **მთავარი გვერდი & Responsive (1 წთ):** დეშბორდის დემონსტრირება დესკტოპსა და მობილურ (375px) რეჟიმში ჰამბურგერ-მენიუს ჩათვლით.
  3. **ძიება და ფილტრაცია (1 წთ):** `useDebounce`-ით ოპტიმიზებული რეალურ დროში ძიების ჩვენება.
  4. **CRUD ოპერაციები & მოდალი (1 წთ):** ახალი ტვირთის დამატება მოდალური ფანჯრის მეშვეობით, Zod ვალიდაციის დემონსტრირება და ბრაუზერის განახლებისას (`Refresh`) მონაცემების შენარჩუნების (`localStorage` persistence) ტესტირება.
  5. **მდგრადობა & შეჯამება (1 წთ):** `ErrorBoundary`-ს მუშაობის ჩვენება და სამომავლო გეგმების განხილვა.
- **სამომავლო განვითარების გეგმა (Version 2.0 Roadmap):**

  1. რეალურ REST API ბექენდთან და მონაცემთა ბაზასთან (PostgreSQL) ინტეგრაცია.
  2. მომხმარებელთა ავტორიზაციისა და როლებზე დაფუძნებული წვდომების (JWT Auth) დამატება.

3. ანალიტიკური გრაფიკებისა და დიაგრამების (Recharts) ინტეგრაცია დეშბორდისთვის.